GEN AI HANDS-ON UNIT 2

SRN: PES2UG23CS089

NAME: ARJUN RAJESH

SEM: 6 SEC: B

```
[9]
0s
from langchain_google_genai import ChatGoogleGenerativeAI

# Model A: The "Accountant" (Precision)
llm_focused = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0.0)

# Model B: The "Poet" (Creativity)
llm_creative = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=2.0)
```

```
[10]
9s
prompt = "Define the word 'Idea' in one sentence."

print("--- FOCUSED (Temp=0) ---")
print(f"Run 1: {llm_focused.invoke(prompt).content}")
print(f"Run 2: {llm_focused.invoke(prompt).content}")

--- FOCUSED (Temp=0) ---
Run 1: An idea is a thought, concept, or mental image formed in the mind.
Run 2: An idea is a thought, concept, or suggestion that is formed or exists in the mind.

[12]
8s
print("--- CREATIVE (Temp=2) ---")
print(f"Run 1: {llm_creative.invoke(prompt).content}")
print(f"Run 2: {llm_creative.invoke(prompt).content}")

--- CREATIVE (Temp=2) ---
Run 1: An idea is a thought or concept that arises in the mind, serving as a basis for understanding, planning, or creation.
Run 2: An idea is a mental impression, thought, concept, or plan formed in the mind, often representing a possibility or a solution.
```

Changing the temperature changes the randomness for the texts generated. When temperature is 0 the content generated from both runs should be the same or very similar and when increased it should vary more as seen here.

```
[7]
1s
from langchain_core.messages import SystemMessage, HumanMessage

# Scenario: Make the AI rude.
messages = [
    SystemMessage(content="You are a rude teenager. You use slang and don't care about grammar."),
    HumanMessage(content="What is the capital of France?")
]

response = llm.invoke(messages)
print(response.content)

Ugh, seriously? Like, why even ask? It's Paris. Duh. Betcha didn't even Google it, did ya? So lazy.
```

The SystemMessage gives personality and constraint control while HumanMessage is the input text for which the prompt has to reply.

Assignment

Create a chain that:

1. Takes a movie name.
2. Asks for its release year.
3. Calculates how many years ago that was (You can try just asking the LLM to do the math).

Try to do it in **one line of LCEL**.

```python
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough



chain = (
    {"movie": RunnablePassthrough()}
    | PromptTemplate.from_template(
        "What is the release year of '{movie}' and how many years ago was that from 2026?"
    )
    | llm
    | StrOutputParser()
)

print(chain.invoke("Inception"))
```

```
The release year of 'Inception' is **2010**.

From 2026, that was **16 years** ago (2026 - 2010 = 16).
```

```python
structured_prompt = """
# Context
You are an HR Manager at a quirky startup called 'RocketBoots'.

# Objective
Write a rejection email to a candidate named Bob.

# Constraints
1. Be extremely brief (under 50 words).
2. Do NOT say 'we found someone better'. Say 'the role changed'.
3. Sign off with 'Keep flying'.

# Output Format
Plain text, no subject line.
"""

print("--- STRUCTURED PROMPT ---")
print(llm.invoke(structured_prompt).content)
```

```
--- STRUCTURED PROMPT ---
Hi Bob,

Thank you for your interest in RocketBoots.

We appreciate you taking the time to interview. However, the role's requirements have changed significantly since your application. We wish you the best in your job search.

Keep flying.
```

The structured prompt helps in generating the content in a particular format or way that we want and reduces the ambiguity in the way the content is generated.

Zero shot is hugely reliant on its training data while few shot gives more context with structure and tone.

Chain of Thought(CoT) is linear reasoning while Tree of Thought (ToT) explores multiple reasoning paths and Graph of Thought (GoT) is more complex compared to the previous two as information can be split, processed and aggregated later.

```python
# 2. CoT Prompt (Magic Phrase)
prompt_cot = f"Answer this question. Let's think step by step. {question}"

print("--- Chain of Thought (Llama3.1-8b) ---")
print(llm.invoke(prompt_cot).content)
```

```
--- Chain of Thought (Llama3.1-8b) ---
To find out how many tennis balls Roger has now, we need to follow these steps:

1. Roger already has 5 tennis balls.
2. He buys 2 more cans of tennis balls. Each can has 3 tennis balls, so he buys 2 x 3 = 6 more tennis balls.
3. Now, we add the tennis balls he already had (5) to the new tennis balls he bought (6). 5 + 6 = 11

So, Roger now has 11 tennis balls.
```

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableParallel, RunnableLambda
from langchain_core.output_parsers import StrOutputParser

problem = "How can I get my 5-year-old to eat vegetables?"

# Step 1: The Branch Generator
prompt_branch = ChatPromptTemplate.from_template(
    "Problem: {problem}. Give me one unique, creative solution. Solution {id}:"
)

branches = RunnableParallel(
    sol1=prompt_branch.partial(id="1") | llm | StrOutputParser(),
    sol2=prompt_branch.partial(id="2") | llm | StrOutputParser(),
    sol3=prompt_branch.partial(id="3") | llm | StrOutputParser(),
)

# Step 2: The Judge
prompt_judge = ChatPromptTemplate.from_template(
    """
    I have three proposed solutions for: '{problem}'

    1: {sol1}
    2: {sol2}
    3: {sol3}

    Act as a Child Psychologist. Pick the most sustainable one (not bribery) and explain why.
    """
)

# Chain: Input -> Branches -> Judge -> Output
tot chain = (
```

```python
tot_chain = (
    RunnableParallel(problem=RunnableLambda(lambda x: x), branches=branches)
    | (lambda x: {**x["branches"], "problem": x["problem"]})
    | prompt_judge
    | llm
    | StrOutputParser()
)

print("--- Tree of Thoughts (ToT) Result ---")
print(tot_chain.invoke(problem))
```

```
--- Tree of Thoughts (ToT) Result ---
Based on the provided solutions, I recommend **Solution 3: Create a "Garden in a Cup"** as the most sustainable approach to encourage a 5-year-old to eat vegetables. Here's w

1. **Involves child in the process**: This solution empowers the child to take an active role in creating their own "Garden in a Cup." By allowing them to choose the vegetabl
2. **Promotes exploration and learning**: The activity encourages the child to explore different textures, colors, and flavors while learning about the various vegetables. Th
3. **Makes mealtime engaging**: By transforming vegetables into a fun and interactive experience, the child is more likely to engage with the meal and try new foods. This app
4. **Fosters healthy eating habits**: By introducing a variety of vegetables in a playful way, the child is more likely to develop a liking for healthy foods. This approach c
5. **Avoids bribery**: Unlike some other solutions, "Create a 'Garden in a Cup'" does not rely on bribes or rewards. Instead, it focuses on creating a positive and engaging e
6. **Encourages independence**: As the child becomes more confident in their ability to create their own "Garden in a Cup," they may be more willing to try new vegetables and

Overall, "Create a 'Garden in a Cup'" is a sustainable approach that encourages children to develop healthy eating habits, explore new foods, and take an active role in their
```

```
graph LR
    Start(Concept) --> A[Draft 1]
    Start --> B[Draft 2]
    Start --> C[Draft 3]
    A & B & C --> Mixer[Aggregator]
    Mixer --> Final[Final Story]
```

```python
# 1. The Generator (Divergence)
prompt_draft = ChatPromptTemplate.from_template(
    "Write a 1-sentence movie plot about: {topic}. Genre: {genre}."
)

drafts = RunnableParallel(
    draft_scifi=prompt_draft.partial(genre="Sci-Fi") | llm | StrOutputParser(),
    draft_romance=prompt_draft.partial(genre="Romance") | llm | StrOutputParser(),
    draft_horror=prompt_draft.partial(genre="Horror") | llm | StrOutputParser(),
)

# 2. The Aggregator (Convergence)
prompt_combine = ChatPromptTemplate.from_template(
    """
    I have three movie ideas for the topic '{topic}':
    1. Sci-Fi: {draft_scifi}
    2. Romance: {draft_romance}
    3. Horror: {draft_horror}

    Your task: Create a new Mega-Movie that combines the TECHNOLOGY of Sci-Fi, the PASSION of Romance, and the FEAR of Horror.
    Write one paragraph.
    """
)
```

```python
    Your task: Create a new Mega-Movie that combines the TECHNOLOGY of Sci-Fi, the PASSION of Romance, and the FEAR of Horror.
    Write one paragraph.
    """
)

# 3. The Chain
got_chain = (
    RunnableParallel(topic=RunnableLambda(lambda x: x), drafts=drafts)
    | (lambda x: {**x["drafts"], "topic": x["topic"]})
    | prompt_combine
    | llm
    | StrOutputParser()
)

print("--- Graph of Thoughts (GoT) Result ---")
print(got_chain.invoke("Time Travel"))
```

```
--- Graph of Thoughts (GoT) Result ---
Here's a paragraph summarizing a potential movie idea that combines the technology of Sci-Fi, the passion of Romance, and the fear of Horror:

"Echoes of Eternity" follows Dr. Emma Taylor, a brilliant physicist who has just cracked the code to manipulating time. As she uses her invention to travel back to the night
```

Vector embeddings of each word represent a feature.
The cosine similarity helps us understand which words are similar by comparing their vector embeddings.